

AWS Project Documentation

Highly Available Multi-Tier Infrastructure with MySQL Replication Using ProxySQL

1. Project Overview

Objective

The objective of this project is to design and deploy a secure, scalable, and highly available infrastructure in AWS using a custom Virtual Private Cloud (VPC). The architecture is designed to host backend applications securely in private subnets, distribute traffic through a load balancer, and implement MySQL database replication for redundancy and reliability.

The environment was deployed in the Mumbai region (`ap-south-1`) and follows a production-style architecture using multiple Availability Zones.

Key Goals

- Secure network segmentation using public and private subnets
 - High availability across multiple Availability Zones (AZs)
 - Secure administrative access using a Bastion Host
 - Controlled internet access for private resources using NAT Gateway
 - Traffic distribution using an Application Load Balancer (ALB)
 - Database redundancy using MySQL Master-Slave replication
 - Database optimization using ProxySQL
-

2. Architecture Summary

Component	Details
AWS Region	Mumbai (<code>ap-south-1</code>)
VPC CIDR	<code>172.32.0.0/16</code>
Total Subnets	6
Public Subnets	3
Private Subnets	3
Subnet CIDR	<code>/20</code>
Backend Instances	3 EC2 Instances
Database Servers	2 MySQL Instances
Database Design	Master-Slave Replication

Component	Details
Database Tool	ProxySQL
Load Balancer	Application Load Balancer
Access Method	Bastion Host

3. VPC Configuration

A custom VPC was created to isolate the project infrastructure and provide controlled networking.

VPC Details

VPC CIDR: 172.32.0.0/16
Region: ap-south-1 (Mumbai)

The `/16` CIDR block was selected to allow enough IP address space for future scalability.

4. Subnet Architecture

A total of **6 subnets** were created and distributed across **3 Availability Zones** for redundancy and high availability.

Public Subnets

Public subnets are used for internet-facing resources.

Purpose

- Application Load Balancer (ALB)
- NAT Gateway
- Bastion Host

Subnet	Availability Zone	Example CIDR
Public Subnet 1	ap-south-1a	<code>172.32.0.0/20</code>
Public Subnet 2	ap-south-1b	<code>172.32.16.0/20</code>
Public Subnet 3	ap-south-1c	<code>172.32.32.0/20</code>

Private Subnets

Private subnets are used for internal application and database servers.

Purpose

- Backend EC2 Instances
- Database Servers

Subnet	Availability Zone	Example CIDR
Private Subnet 1	ap-south-1a	172.32.48.0/20
Private Subnet 2	ap-south-1b	172.32.64.0/20
Private Subnet 3	ap-south-1c	172.32.80.0/20

Design Consideration

Separating public and private resources improves security by ensuring backend and database resources are not directly exposed to the internet.

5. Internet Gateway (IGW)

An Internet Gateway (IGW) was created and attached to the VPC.

Purpose

The IGW enables communication between internet-facing resources and the internet.

Route Configuration

The public route table contains the following route:

0.0.0.0/0 → Internet Gateway

Associated Resources

The route table was associated with all public subnets.

Result

The following components can communicate with the internet:

- Bastion Host
 - Application Load Balancer
 - NAT Gateway
-

6. Route Table Configuration

Public Route Table

Configured to send internet traffic to the Internet Gateway.

```
Destination: 0.0.0.0/0
Target: Internet Gateway
```

Associated with:

- Public Subnet 1
- Public Subnet 2
- Public Subnet 3

Private Route Table

Private subnets require outbound internet access for software updates and package downloads.

The private route table was configured to use a NAT Gateway.

```
Destination: 0.0.0.0/0
Target: NAT Gateway
```

Associated with:

- Private Subnet 1
 - Private Subnet 2
 - Private Subnet 3
-

7. NAT Gateway

A NAT Gateway was deployed in a public subnet.

Purpose

The NAT Gateway provides outbound internet access for instances inside private subnets while preventing direct inbound access from the internet.

Use Cases

- Operating system updates
- Package installation
- Application dependency downloads
- External API communication

Traffic Flow

Private EC2 → NAT Gateway → Internet Gateway → Internet

Security Benefit

Private servers remain inaccessible from the public internet.

8. Bastion Host Configuration

A Bastion Host (Jump Server) was deployed in a public subnet.

Purpose

The Bastion Host provides secure SSH access to private instances.

Since backend and database servers are deployed in private subnets, direct internet access is disabled.

SSH Flow

```
Administrator Laptop
    ↓
Bastion Host
    ↓
Private EC2 / Database Servers
```

Benefits

- Improved security
 - No public IP required for private servers
 - Controlled administrative access
-

9. Backend EC2 Infrastructure

Three backend EC2 instances were deployed across three Availability Zones.

Instance	Availability Zone
Backend-1	ap-south-1a
Backend-2	ap-south-1b
Backend-3	ap-south-1c

Purpose

These instances host the backend application.

High Availability Strategy

Distributing instances across multiple AZs ensures application availability even if one Availability Zone becomes unavailable.

10. Target Group Configuration

A Target Group was created and all backend EC2 instances were registered.

Purpose

The Target Group routes incoming traffic to healthy backend instances.

Health Checks

The load balancer continuously monitors the health of backend servers.

If a server becomes unhealthy:

- Traffic is automatically stopped to that server.
 - Requests are routed only to healthy instances.
-

11. Application Load Balancer (ALB)

An Application Load Balancer (ALB) was deployed to distribute application traffic.

Purpose

The ALB distributes incoming requests across multiple backend EC2 instances.

Request Flow

```
graph TD;
    A[User Request] --> B[Application Load Balancer];
    B --> C[Target Group];
    C --> D[Healthy Backend Instance];
```

Benefits

- Load balancing
 - High availability
 - Automatic failover
 - Health monitoring
-

12. Database Architecture

Two MySQL database servers were deployed inside private subnets.

Database Server	Role
DB-1	Master
DB-2	Slave

Security Design

Database servers were intentionally placed in private subnets to restrict public access.

Benefits

- Increased database security
 - Internal-only communication
 - Reduced attack surface
-

13. MySQL Master-Slave Replication

MySQL replication was configured between the two database servers.

Replication Model

```
Master Database
    ↓ Replication
Slave Database
```

Working Principle

1. Application writes data to the Master Database.
2. Changes are replicated to the Slave Database.
3. Slave remains synchronized with the Master.

Example Operations

Write operations:

INSERT
UPDATE
DELETE

These are executed on:

Master Database

The replicated data becomes available on:

Slave Database

Benefits

- Data redundancy
- Backup availability
- Improved reliability
- Read scalability

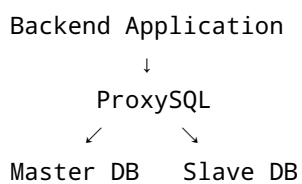
14. ProxySQL Configuration

ProxySQL was implemented to manage MySQL traffic and optimize database operations.

Purpose

ProxySQL acts as a middleware layer between backend servers and database servers.

Architecture



Query Routing

Write Queries

INSERT
UPDATE
DELETE

Sent to:

Master Database

Read Queries

SELECT

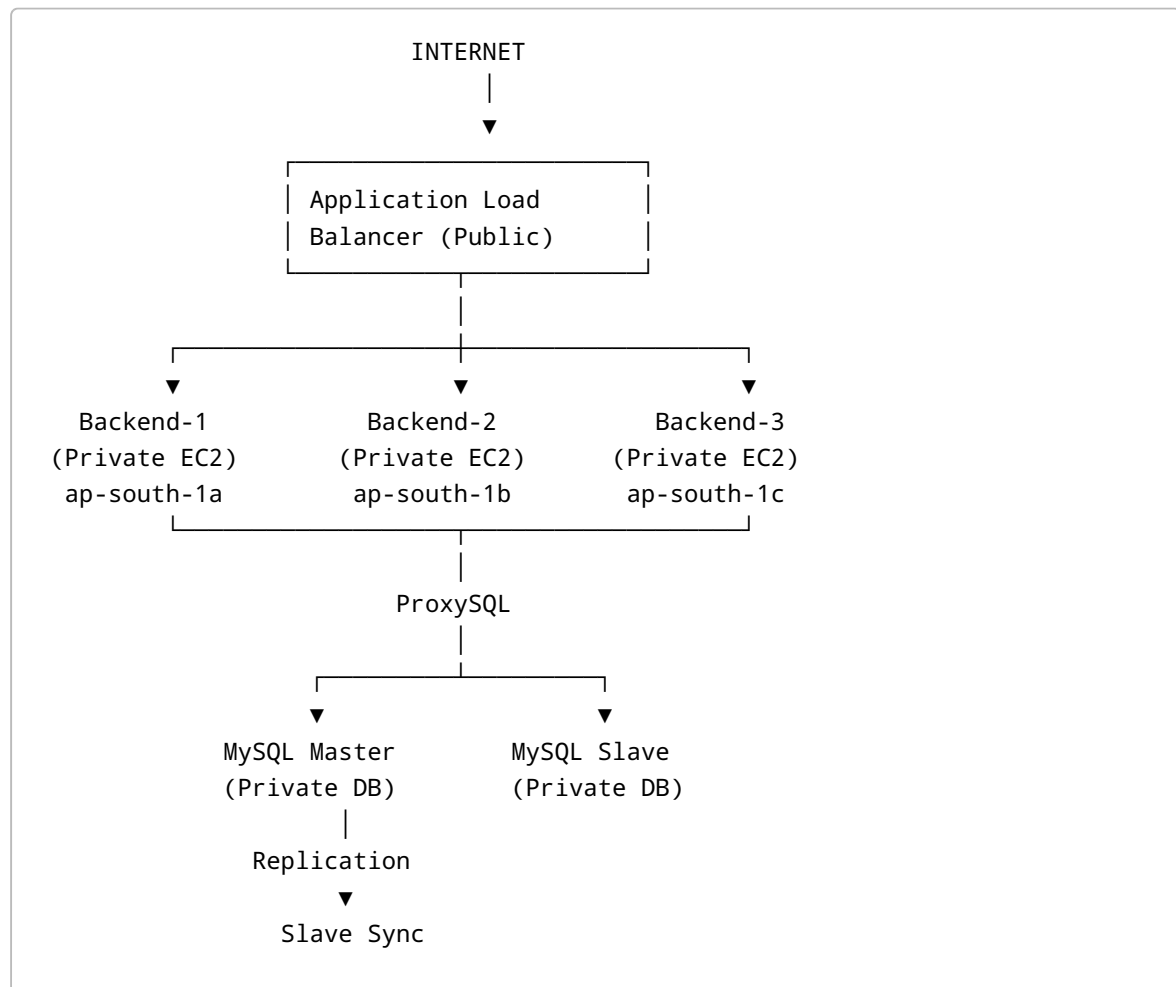
Can be directed to:

Slave Database

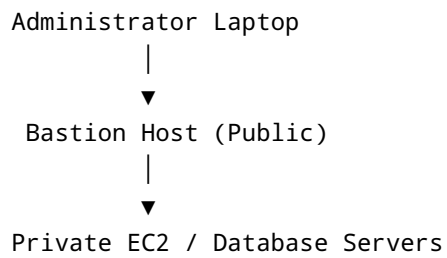
Benefits

- Read/Write splitting
- Better database performance
- Connection pooling
- Query optimization

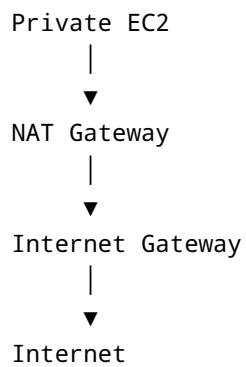
15. Complete Infrastructure Diagram



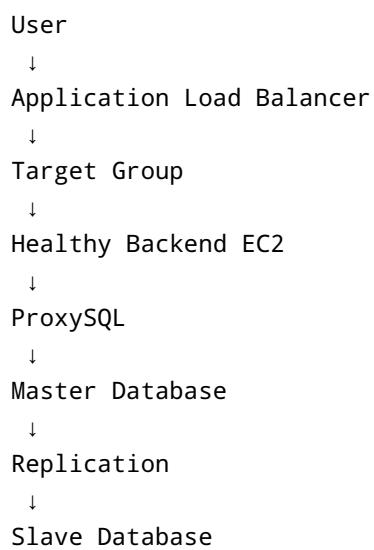
SSH Access Path



Private Internet Access



16. End-to-End Request Flow



17. Security Best Practices Implemented

- Backend and databases deployed in private subnets
 - Bastion Host used for secure administrative access
 - NAT Gateway used for controlled outbound access
 - No direct public access to database servers
 - Multi-AZ deployment for high availability
 - Load balancing for fault tolerance
-

18. Conclusion

This project demonstrates a production-style AWS architecture that follows modern cloud infrastructure best practices.

The deployment includes secure networking, private compute resources, load balancing, high availability, and replicated database architecture.

The solution is scalable, fault tolerant, and secure for hosting backend workloads in AWS.